AR Sudoku: Vision and Solver Design

Leonardo Tisato

1. Scope

The user points the camera at a printed Sudoku, empty or already started by hand. The app writes the missing digits on the paper and marks the user's wrong digits.

- **My part (vision + solver):** from a frame or photo to grid position, digits read (printed and handwritten), confidence, solution, reveal order and wrong digits.
- **Ali's part (app):** camera, screens, drawing the digits on the paper, Check screen, hosting.

2. Interface

A Python package `sudoku_cv` that runs inside the browser with Pyodide, in a background thread. It only uses `numpy` and Pyodide's OpenCV build (4.x), no threads and no windows. The digit templates and the handwritten-digit training data are small files shipped with the package.

- `tracker = SudokuTracker()`: one object per camera stream. It keeps state between frames (the puzzle read so far).
- `result = tracker.process_frame(frame, debug=False)`: `frame` is a `uint8` numpy array, RGBA ($H \times W \times 4$, straight from a canvas) or BGR ($H \times W \times 3$). The app shrinks it to 640 px first.
- `result = process_image(image, debug=False)`: same output for a single photo (upload, sample). No waiting for a stable grid, no state.
- `tracker.set_givens(givens)`: the user fixed digits on the Check screen. The tracker keeps the corrected digits, solves again and returns the new result. The corrected digits stay until `reset()`.
- `solve(givens, user_digits)`: the solver alone, returns the `status`, `solution`, `order` and `wrong` fields below.
- `tracker.reset()`: forget the current puzzle ("new puzzle").

`result` is a plain dict of lists, ints, floats and strings:

```
{"status": "solved",                # no_grid | reading | no_solution | solved
 "corners": [[x,y], ...],           # 4 floats, frame pixels, order TL, TR, BR, BL
 "homography": [[...], ...],        # 3x3, square (0..450) -> frame
 "givens": [[5,3,0,...], ...],      # 9x9, printed digits, 0 = none
 "user_digits": [[0,0,4,...], ...], # 9x9, handwritten digits, 0 = none
 "confidence": [[0.97,...], ...],   # 9x9, 0..1, how sure the reading of each cell is
 "solution": [[5,3,4,...], ...],    # 9x9, null unless status is solved
 "order": [[r,c], ...],             # cells with no digit at all, in reveal order
 "wrong": [[r,c], ...],             # handwritten digits that differ from the solution
 "debug": {...}}                    # only if debug=True: threshold, contour, flat grid
```

- Coordinates: x = column, y = row, origin top-left, in pixels of the frame passed in. Rows and columns of the puzzle are 0–8 from the top-left of the puzzle.
- Status: `no_grid` = no puzzle in view; `reading` = puzzle found, hold still; `no_solution` = the printed digits read make the puzzle unsolvable (probably a misread, go to Check); `solved` = start or continue the reveal. Wrong handwritten digits do not block the solution, they go in `wrong`.
- Reveal order: the cell with the fewest candidates comes first, so a single step also works as a hint. The app draws each digit at the cell center in the 450×450 square and maps it with `homography`.

A stub with the same interface and fake data comes first, so the app can be built in parallel.

3. Pipeline

1. **Grid detection:** grayscale, Gaussian blur, adaptive threshold (handles shadows), largest contour that approximates to 4 corners. Alternative to compare: Canny edges + Hough lines, keep the outermost horizontal and vertical lines, intersect them.
2. **Rectification:** homography from the 4 corners to a 450×450 square.
3. **Cells:** split into 81 cells of 50×50, cut the borders to remove grid lines. A cell is empty if the ink pixels in its center are below a threshold.
4. **Printed digits:** the largest ink blob is centered and resized, then compared by normalized cross-correlation (template matching) to digits 1–9 rendered from a few common fonts. The best score is the confidence.
5. **Handwritten digits:** a cell with ink but a low template score is handwritten. It is classified with HOG features and k-nearest neighbours, trained on MNIST plus a few handwritten cells of our own. The share of neighbours that agree is the confidence.
6. **Solver (by hand):** check the printed digits for duplicates in rows, columns and boxes, then backtracking that fills the cell with the fewest candidates first. It gives the reveal order. Handwritten digits are compared with the solution to find the wrong ones.

4. Live behaviour

- Reading and solving run when the grid has been found in a few consecutive frames with little corner motion. The result is cached. While the grid stays stable, the cells are read again every few seconds, so new pencil digits are picked up.
- Every frame only re-detects the 4 corners and smooths them over time (moving average). If the grid is lost for a short time, the last result is kept. If it is lost for longer, or the printed digits read differ from the cache, the tracker resets.
- Targets in the browser at 640 px: the per-frame path fast enough for Ali's 15 updates/s on a laptop and 10 on phones; reading + solving about 1 s.

5. Assumptions and limits

- One standard 9×9 puzzle in view, roughly upright (less than about 45°).
- Faint pencil, messy handwriting, curved paper and glare may fail: reported as limitations.

6. Evaluation (for the report)

Our own labelled videos and photos (tilt, shadows, glare, curved paper, fonts, half filled). Metrics: grid detection rate (contour vs Hough), printed and handwritten digit accuracy, wrong digits caught, solved rate, FPS, jitter.

7. Repo and working rules

My code is in `sudoku_cv/`, the app in `app/`, sample photos (printed and half filled) in `samples/`. We do not edit each other's folders; interface changes are agreed first.

8. Timeline

- **Oct 28:** interface stub pushed (fake data, all fields).
- **Nov 2:** Ali decides if Pyodide is fast enough. If not, the same interface runs on a server.
- **Nov 15:** real pipeline for printed digits behind the same interface.
- **Nov 16 – 22:** integration with the app; handwritten digits added.
- **Nov 30:** code freeze. **Dec 1/3:** presentation. **Dec 11:** report.